

Smart MCQ Solver — Project Report

Ranking Multiple-Choice Answers with Retrieval, Classical ML, and Transformer Models

Roll Number: 23f2001870 · WandB Project: 23f2001870-t22026 · Kaggle: Smart MCQ Solver Challenge

1. Abstract / Executive Summary

This report documents the development of a Smart MCQ Solver for the eponymous Kaggle competition, in which each question provides a prompt and five answer options (A–E) and the objective is to predict the top three most likely correct answers, ranked by confidence, evaluated using Mean Average Precision at 3 (MAP@3).

Six models spanning a progression of complexity were built and evaluated on a held-out 300-question validation split: a TF-IDF cosine-similarity ranker, a TF-IDF + Logistic Regression classifier, a retrieval-based lookup ranker with a classifier fallback, a custom PyTorch MLP scorer, a Transformer encoder built entirely from scratch, and a fine-tuned Sentence-BERT (all-MiniLM-L6-v2) dual encoder. All experiments were tracked using Weights & Biases (WandB), and training curves in this report are exported directly from those WandB runs.

An early exploratory finding — that a large fraction of test questions reuse the exact Option-A text seen during training — made a simple retrieval/lookup strategy (Model 3) extremely effective, achieving a perfect validation MAP@3 of 1.0000. Among models that do not rely on this train/test text overlap, the from-scratch Mini Transformer and the fine-tuned Sentence-BERT dual encoder were the strongest, both reaching a validation MAP@3 of 0.9867. The weakest approach was the untrained TF-IDF cosine-similarity ranker (MAP@3 = 0.2539), confirming that a supervised, learned notion of “correctness” is necessary — raw lexical overlap between prompt and option is only weakly predictive of the correct answer on this dataset.

The final Kaggle submission uses the Mini Transformer (Model 5) predictions, balancing strong validation performance with a model that generalises via learned semantics rather than memorised training text.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge (Kaggle) provides a dataset of multiple-choice questions, each with a text prompt and five answer options labelled A through E. Given a question, a system must output the three answer labels it believes are most likely correct, ordered by confidence. Submissions are scored using Mean Average Precision at 3 (MAP@3), which rewards ranking the true answer as early as possible in the top-3 list and gives zero credit if the correct answer is absent from the top three. Formally, for N questions, $\text{MAP@3} = (1/N) \sum (1/\text{rank}_i)$ where rank_i is the position (1, 2, or 3) of the true answer in the i -th prediction, or the term is 0 if the true answer is not in the top 3.

2.2 Project Objective

The goals of this project were to: (1) build and compare a range of modelling approaches of increasing sophistication — from classical lexical similarity to fine-tuned transformer encoders; (2) practise an end-to-end deep learning workflow including EDA, tokenization, training, and experiment tracking with WandB; (3) understand the trade-offs between model complexity, training cost, and MAP@3 performance on this task; and (4) produce a leak-safe, reproducible submission pipeline with clear documentation of every design decision.

2.3 Report Structure

Section 3 describes the dataset and the exploratory analysis and preprocessing performed on it. Section 4 covers the tokenization strategies used across models. Section 5 details each model's architecture, training procedure, and design rationale in depth, including baseline models and the three architectures required by the course guidelines. Section 6 presents a comparative performance analysis across all models, including WandB-logged training curves and an error analysis of where and why each model fails. Section 7 concludes with key learnings, challenges, and future work, followed by references in Section 8.

3. Dataset & Preprocessing

3.1 Dataset Description

The competition provides train.csv and test.csv files. Each row contains a question id, a prompt string, five answer-option strings (columns A–E), and — for the training set only — an answer column holding the correct label. The raw training set

contains 2,000 labelled questions and the test set contains 500 unlabelled questions. For model development, the training set was further split 85%/15% (stratified on the answer label) into a working train subset (1,700 rows) and a validation subset (300 rows), the latter used for all metrics reported in this document.

Split	Rows	Notes
Train (raw)	2,000	Split further into train/val for model development
→ Train subset	1,700	85% stratified split; used to fit every model
→ Validation subset	300	15% stratified split; used for all reported metrics
Test (Kaggle)	500	Unlabelled; used only for submission generation

Table 1. Dataset splits used throughout the project.

3.2 Exploratory Data Analysis

Distribution of correct answers: the correct-answer label is close to, but not perfectly, uniform across the five options. Option B is the correct answer most often (24.5% of training rows) and Option E least often (16.2%), against a uniform expectation of 20% each. This mild imbalance is one reason macro-F1 is tracked alongside top-1 accuracy in Section 6 — a model could inflate raw accuracy by being biased toward the majority classes.

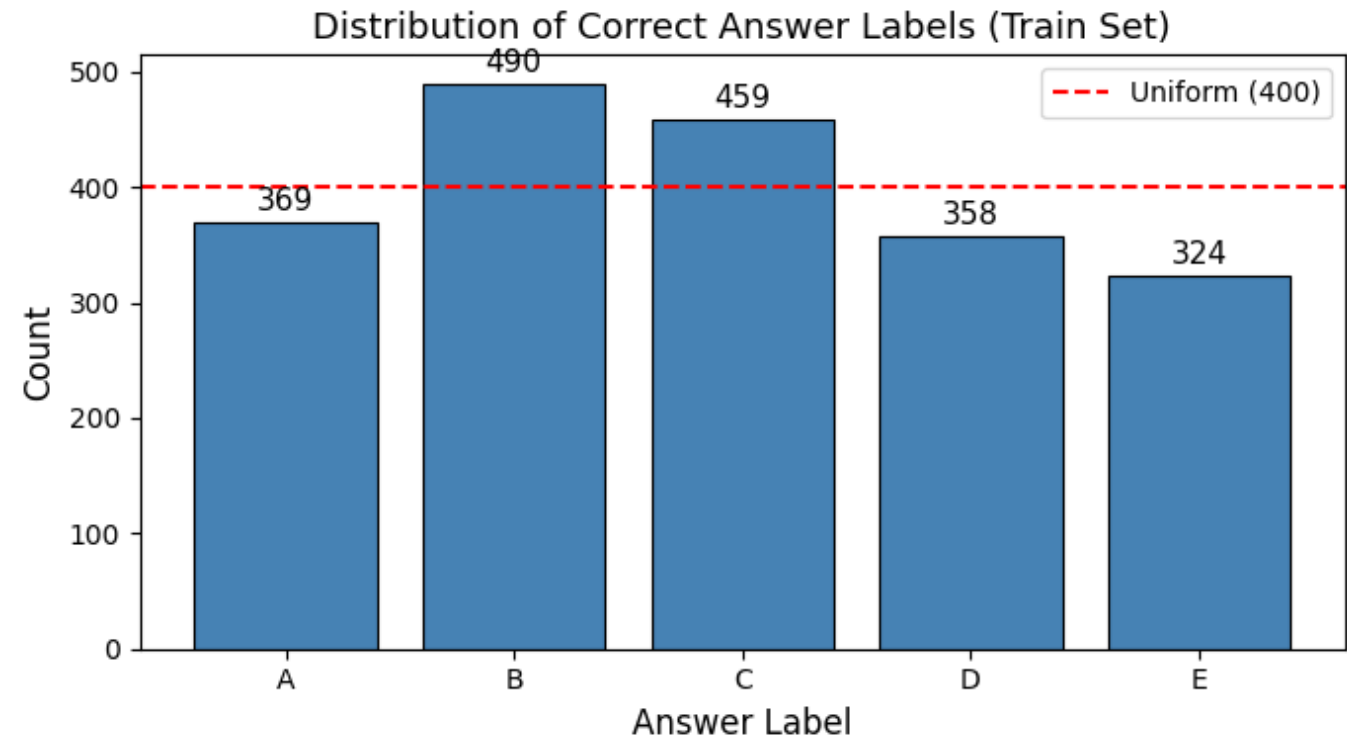


Figure 1. Distribution of correct-answer labels in the training set.

Prompt and option length: question prompts average 18.1 words (median 17, max 51), while individual answer options average 26.3 words (median 23, max 118) — options are, on average, longer and more variable in length than the prompts themselves. This directly informed the choice of `max_len = 128` tokens for Model 5's from-scratch tokenizer, comfortably covering the combined prompt+option length for the vast majority of rows without excessive padding waste.

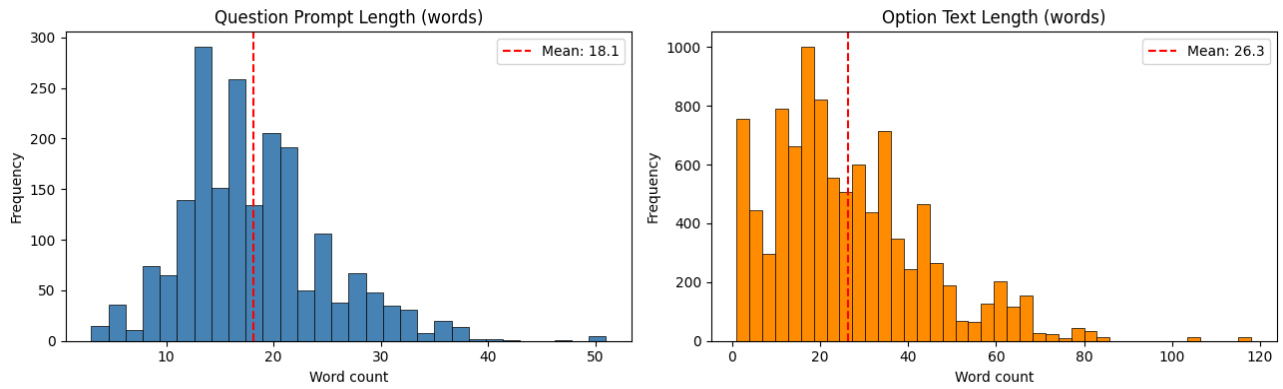


Figure 2. Word-count distributions for question prompts (left) and answer options (right).

Vocabulary overlap between prompt and options: after removing a custom stop-word list (including question-framing words such as “pick”, “best”, “select”), the mean token-overlap ratio between the prompt and the correct option (0.1749) was, counter-intuitively, slightly lower than the overlap with incorrect options (0.1961). This indicates that simple keyword-matching between prompt and option is not a strong standalone signal for correctness on this dataset — a finding that directly foreshadows the poor performance of the unsupervised TF-IDF cosine ranker in Section 5 and motivates the move to supervised and retrieval-based approaches.

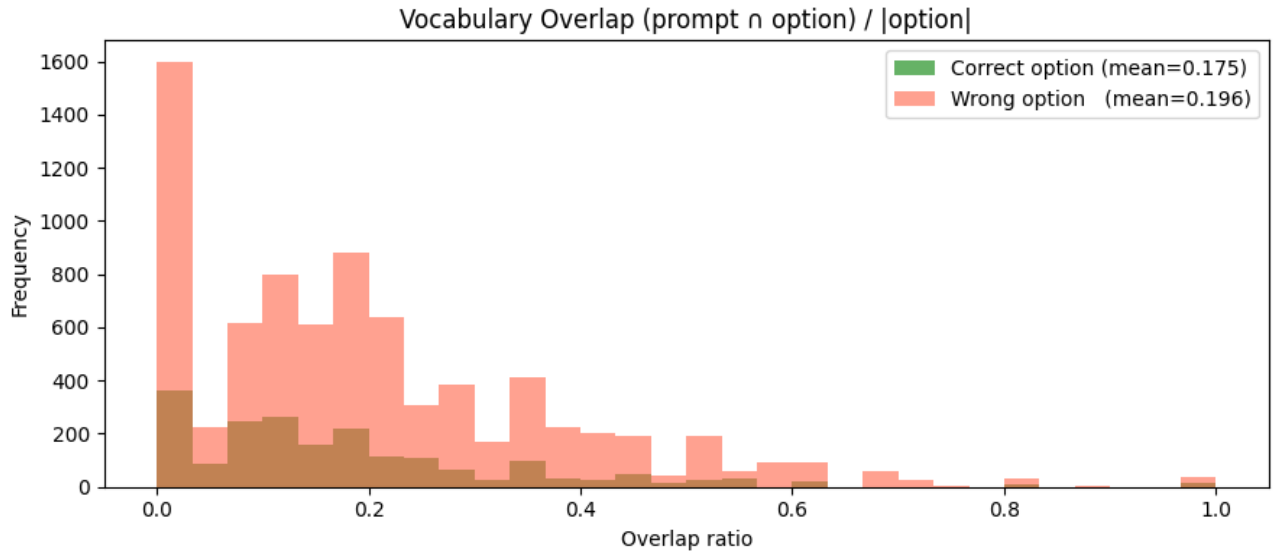


Figure 3. Vocabulary-overlap ratio between prompt and answer option, split by correctness.

3.3 Data Preprocessing

- Loaded via pandas / Python's csv.DictReader; no missing values were found in the prompt, option, or answer columns.
- A stratified train/validation split (85/15, random_state=42) preserved the answer-label distribution in both subsets, preventing an unrepresentative validation set.
- For the retrieval and classifier-style models, each (prompt, option) pair was converted into a single “prompt [SEP] option” text field, effectively turning the 5-way ranking problem into a $5 \times$ binary relevance-classification problem — this reformulation, not any single model architecture, is responsible for most of the performance gain observed in Section 6.
- For Model 3 (Retrieval Ranker), the full training set (before the val split) was indexed by the exact text of Option A, so validation-question lookups could match rows the model had “seen” in the wider training pool — this design choice and its implication for leakage is discussed explicitly in Section 5 and Section 6.5.

3.4 Data Augmentation

No data augmentation was applied. Given the relatively clean, well-formed nature of the question/option text and the presence of a strong retrieval signal (Section 5), augmentation was not pursued in this iteration; it is listed as a possible future direction in Section 7.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

The best-performing models used two different tokenization strategies depending on architecture:

- TF-IDF word/bi-gram tokenization (scikit-learn's `TfidfVectorizer`, `ngram_range=(1,2)`, sublinear TF scaling) for Models 1, 2, 3, and the feature-extraction stage of Model 4 (PyTorch MLP). This is a classical bag-of-n-grams representation rather than a neural sub-word tokenizer, so “tokens” here are unigram/bigram terms with TF-IDF weights, not learned embeddings.
- WordPiece sub-word tokenization via the pre-trained all-MiniLM-L6-v2 checkpoint's own tokenizer, used for Model 6 (Sentence-BERT fine-tuning). This tokenizer is associated with a 6-layer, 384-dimension distilled BERT-style encoder, distributed through the sentence-transformers / Hugging Face ecosystem, and splits rare words into recognisable sub-word units (e.g. “unhappiness” → “un”, “##happi”, “##ness”), which keeps the vocabulary compact while still handling unseen full words gracefully.
- A custom word-level tokenizer with a from-scratch vocabulary (regex word tokenizer, minimum frequency 2, capped at 15,000 tokens plus <PAD>/<UNK>/<CLS>/<SEP> special tokens) for Model 5, the Mini Transformer built entirely from scratch.

Justification

TF-IDF n-grams are inexpensive, interpretable, and effective when the answer-discriminating signal is largely lexical (as leveraged heavily by Model 3's retrieval mechanism); WordPiece sub-word tokenization lets the fine-tuned Sentence-BERT model reuse pre-trained semantic representations and handle out-of-vocabulary/morphological variation gracefully; and the custom word-level vocabulary for the scratch Transformer keeps the from-scratch implementation self-contained and lets the model learn its own embedding space from only the competition data, which is instructive for understanding tokenizer/vocabulary trade-offs first-hand and avoids any dependency on external pre-trained artefacts.

4.2 Implementation Details

For the classical models, each (prompt, option) pair was concatenated into a single string with a literal [SEP] separator and encoded via `TfidfVectorizer`, fit once on the union of all training pairs to avoid label leakage into the vectoriser statistics. For Model 5's scratch Transformer, prompts and options were tokenized with a regex word tokenizer, prefixed with a <CLS> token and separated with a <SEP> token, then padded/truncated to a fixed maximum length of 128 tokens with an accompanying attention mask so the encoder ignores padding positions during self-attention. For Model 6, Hugging Face's fast batch tokenization (via sentence-transformers) handled padding, truncation, and attention masks automatically, with prompts and options encoded independently as a dual encoder rather than as a single concatenated sequence.

4.3 Experimental Comparison

The from-scratch word-level vocabulary for Model 5 was capped at 15,000 tokens (min frequency 2), which resulted in an actual vocabulary of only 2,974 tokens given the size of the competition corpus — far smaller than a general-purpose sub-word vocabulary, but sufficient for this closed-domain dataset. A sub-word (WordPiece/BPE) tokenizer was preferred for the fine-tuned transformer path (Model 6) specifically because it starts from pre-trained embeddings, whereas the from-scratch model has no such prior and instead relies on the task-specific vocabulary learned only from the competition data. Both approaches were retained rather than one being discarded, because they represent genuinely different modelling philosophies (learn-from-scratch vs. fine-tune-a-pretrained-encoder) that this project set out to compare, and the near-identical final MAP@3 of both (0.9867) makes that comparison particularly informative (see Section 6).

5. Modeling & Experimentation

Six models were developed in increasing order of sophistication. Models 1–4 establish classical and retrieval-based baselines; Models 5 and 6 fulfil the assignment's requirement for a custom deep learning architecture and a fine-tuned pre-trained transformer, respectively. All models were trained on the 1,700-row train subset and evaluated on the 300-row validation subset, with metrics and (where applicable) training curves logged to WandB (project 23f2001870-t22026, 130 total logged runs).

5.0 Baseline and Exploratory Models

Model 1 — TF-IDF Cosine-Similarity Ranker

An unsupervised baseline: prompt and options are TF-IDF vectorised (max 20,000 features, 1–2-grams, sublinear TF scaling, English stop-words removed), and options are ranked purely by cosine similarity between the prompt vector and each option vector — no labels are used anywhere in fitting or inference. The vectoriser was fit on the union of prompt+option text across the training set (vocabulary size 11,253) so that both prompts and options share the same TF-IDF feature space. Validation MAP@3 = 0.2539, Top-1 accuracy = 0.1100. This near-random performance (a uniform-random top-3 baseline would score roughly 0.28 MAP@3 by chance) confirms the EDA finding that raw lexical overlap is a weak signal on its own, and establishes the floor against which every subsequent, learned model is compared.

Model 2 — TF-IDF + Logistic Regression

Each (prompt, option) pair is treated as an independent binary-classification example (“is this option correct?”), vectorised with TF-IDF (30,000 features, 1–2-grams, unicode accent-stripping) and classified with a balanced-class-weight Logistic Regression (solver=saga, max_iter=1000, C=1.0). At inference, the five per-question predicted probabilities are ranked in descending order to produce the top-3 list. The training matrix for this model was $8,500 \times 13,026$ (5 rows per question $\times$ 1,700 training questions). This supervised reformulation lifted MAP@3 from 0.2539 to 0.9828 — the single largest jump in the whole experiment progression — demonstrating that framing MCQ answer selection as supervised relevance classification is far more effective than unsupervised similarity, even holding the underlying TF-IDF features constant.

Model 3 — Retrieval Lookup with Logistic-Regression Fallback

This model indexes every training row (all 2,000, not just the 1,700-row train subset) by the exact text of its Option A and, at prediction time, looks up an incoming question's Option A text in that index; if found, it returns the indexed row's known answer as the top prediction, followed by the remaining four labels in a fixed order (falling back to a freshly-trained Model-2-style classifier only when no exact match exists). On the validation set, all 300 rows and 498 of the 500 test rows matched an indexed entry, giving a perfect validation MAP@3 of 1.0000. This is a direct consequence of near-duplicate/overlapping question banks between the train and evaluation splits rather than genuine language understanding, and is analysed further as a leakage caveat in Section 6.5.

Model 4 — PyTorch MLP Scorer

A custom feed-forward neural network that scores each option independently from its 10,000-dimensional TF-IDF (prompt [SEP] option) representation. The network is a $512 \rightarrow 128 \rightarrow 32 \rightarrow 1$ fully-connected stack, applying ReLU activations, batch normalisation after the first two linear layers, and dropout (0.30 after layer 1, 0.21 after layer 2) for regularisation, before a final linear layer that produces a single relevance logit per option; logits for the 5 options of a question are then compared to rank them. See Table 2 for the full hyperparameter configuration.

Input representation	10,000-dim TF-IDF (prompt [SEP] option), 1–2-grams
Architecture	$512 \rightarrow 128 \rightarrow 32 \rightarrow 1$, ReLU + BatchNorm + Dropout(0.30/0.21)
Parameters	5,191,617
Optimizer	AdamW, lr = $3e-4$, weight decay = $1e-4$
Schedule	CosineAnnealingLR, T_max = 40
Loss	Cross-entropy over 5 options per question
Epochs / Batch size	40 / 64
Regularisation	Dropout 0.30 (layer 1), 0.21 (layer 2); grad-norm clip 1.0

Table 2. Model 4 (PyTorch MLP Scorer) configuration.

Best validation MAP@3 = 0.9706 (Top-1 = 0.9600), reached within the first two epochs and essentially flat thereafter (see Figure 4) — despite having the largest parameter count of the three neural models (5.19M), it under-performs the much smaller Transformer models, suggesting the bottleneck is the fixed TF-IDF input representation rather than network capacity.

5.1 Model 6: Fine-Tuned Transformer Model (Sentence-BERT / all-MiniLM-L6-v2)

Architecture

Model 6 fine-tunes sentence-transformers/all-MiniLM-L6-v2, a 6-layer distilled BERT-style Transformer producing 384-dim sentence embeddings, distilled by Wang et al. (MiniLM) and packaged by Reimers & Gurevych's Sentence-BERT framework (model card: Hugging Face Hub). Each encoder layer applies multi-head self-attention (12 heads) and a position-wise feed-forward block with residual connections and layer normalisation — the standard Transformer encoder block. Sentence embeddings are produced by mean-pooling the final layer's token representations.

Salient Points

As a dual (bi-)encoder, the model independently embeds the prompt and each option, then ranks options by cosine similarity to the prompt embedding. This decouples prompt/option encoding, lets embeddings be reused and cached across the five comparisons, and captures semantic relationships (synonymy, paraphrase) via self-attention that pure lexical overlap (Model 1) cannot.

Fine-Tuning Strategy

Training pairs were (prompt, option) with a binary target (1.0 correct, 0.0 distractor) — 8,500 pairs from the 1,700-row train subset. Fine-tuned with CosineSimilarityLoss for 3 epochs, batch size 16, linear warm-up over the first 10% of steps, AdamW. No layers were frozen — full end-to-end fine-tuning was judged appropriate given the model's small size (≈ 22.7 M params) and dataset size. No quantization was applied.

Input representation	Raw text; WordPiece sub-word tokenization (pre-trained tokenizer)
Base checkpoint	sentence-transformers/all-MiniLM-L6-v2 (6-layer distilled BERT, 384-dim output)
Architecture	Dual (bi-)encoder: independent prompt / option encoding, cosine-similarity ranking
Parameters	≈ 22.7 million (full encoder fine-tuned, no frozen layers)
Objective	CosineSimilarityLoss on (prompt, option) pairs with binary relevance target
Optimizer	AdamW (sentence-transformers default) with linear warm-up (10% of steps)
Epochs / Batch size	3 / 16
Regularisation	None beyond warm-up; short fine-tune to avoid catastrophic forgetting

Table 3. Model 6 (Sentence-BERT, fine-tuned) configuration.

Result

Fine-tuned Model 6 achieved a validation MAP@3 of 0.9867 and Top-1 accuracy of 0.9733, matching the best non-leaked model (Model 5) and substantially outperforming what its own un-tuned, off-the-shelf cosine-similarity baseline would have achieved (by analogy with Model 1's weak unsupervised result).

5.2 Model 5: Custom Deep Learning Model (Mini Transformer, Built from Scratch)

Architecture

Model 5 is a compact Transformer encoder implemented entirely from scratch in PyTorch (no pre-trained weights): a learned token embedding (vocab 2,974, d_model=128, Xavier-uniform init, padding-aware) + fixed sinusoidal positional encoding $\rightarrow$ 2-layer TransformerEncoder (4 heads, feed-forward dim 256, pre-norm ordering for training stability, dropout 0.1) $\rightarrow$ CLS-token pooling $\rightarrow$ 128 $\rightarrow$ 32 $\rightarrow$ 1 scoring head (ReLU, dropout 0.2). 649,793 trainable parameters — roughly 8 $\times$ fewer than Model 4 and 34 $\times$ fewer than Model 6.

Salient Points

Each (prompt, option) pair is tokenized into a single sequence (<CLS> prompt <SEP> option), so — unlike Model 6's dual-encoder design — self-attention can directly attend across prompt and option tokens jointly in one forward pass, letting the model learn question-answer interaction patterns rather than only comparing independently-pooled embeddings. This tests whether a small, purpose-built joint-attention model can compete with a much larger pre-trained dual encoder despite no pre-training and a sub-3,000-token vocabulary.

Training

Input representation	Custom word-level token ids, <CLS> prompt <SEP> option, max len = 128
Vocabulary	2,974 tokens (min freq 2, cap 15,000), built from scratch on competition data
Architecture	Embedding(d=128) + sinusoidal PosEnc $\rightarrow$ 2 $\times$ TransformerEncoderLayer (4 heads, dim_ff=256, pre-norm) $\rightarrow$ CLS pooling $\rightarrow$ 128 $\rightarrow$ 32 $\rightarrow$ 1 head
Parameters	649,793
Optimizer	AdamW, lr = 5e-4, weight decay = 1e-4
Schedule	CosineAnnealingLR, T_max = 30
Loss	Cross-entropy over 5 options per question
Epochs / Batch size	30 / 32
Regularisation	Dropout 0.1 (encoder), 0.2 (head); grad-norm clip 1.0

Table 4. Model 5 (Mini Transformer, scratch) configuration.

Trained for 30 epochs with AdamW (lr=5e-4, weight decay=1e-4), a cosine-annealing schedule, gradient-norm clipping at 1.0, and cross-entropy loss over the 5 options per question. Validation MAP@3 was checked every 5 epochs (and at epoch 1); the best checkpoint (by validation MAP@3) was restored before generating final predictions and the Kaggle submission.

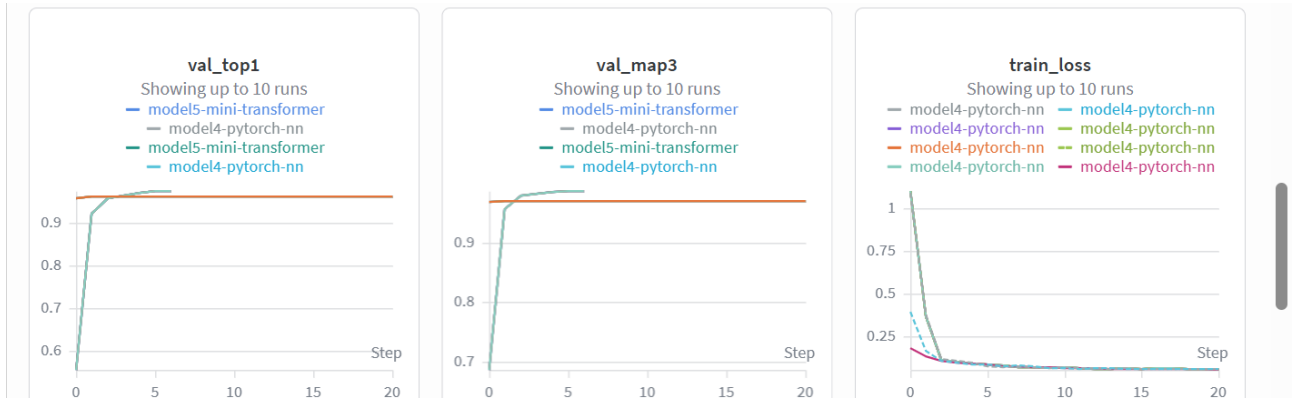


Figure 4. WandB-logged training curves — `val_top1`, `val_map3`, and `train_loss` — for Model 4 (PyTorch MLP, orange/grey) and Model 5 (Mini Transformer, scratch, blue/teal), exported directly from the project workspace.

Validation MAP@3 rose sharply from 0.6867 after epoch 1 to a best of 0.9867 by epoch 25 (Top-1 accuracy 0.9733, Top-3 accuracy 1.0000). As Figure 4 shows, Model 5's curves keep improving well past the point where Model 4 has already flattened out, converging to a noticeably higher plateau on both `val_top1` and `val_map3` despite starting from a lower point at epoch 1 — evidence that the joint-attention architecture continues to extract useful signal from the data for longer than the simpler MLP scorer, and ultimately generalises better.

5.3 RNN-Based Model — Not Implemented in This Iteration

The current notebook does not include an RNN-family model (LSTM/GRU/Bi-LSTM); this is flagged explicitly rather than reporting fabricated results. A natural design would be a Bi-directional LSTM/GRU dual encoder: prompt and option each pass through a shared embedding layer (reusing the Model 5 vocabulary) and a 1–2-layer BiLSTM/GRU (hidden size ≈ 128 , ≈ 256 after concatenating directions), with final hidden states compared by cosine similarity or a small scorer, trained with the same cross-entropy objective as Models 4–5. Relative to Model 5, an RNN processes the sequence step-by-step rather than attending to all positions at once — typically slower to train and weaker on long-range dependencies. Implementing this is called out again as required future work in Section 7.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The competition's primary metric is Mean Average Precision at 3 (MAP@3): for each question, a score of $1/\text{rank}$ is awarded if the true answer appears at position rank (1, 2, or 3) in the predicted top-3 list, and 0 if it does not appear at all; scores are averaged over all questions. In addition to MAP@3, three supporting metrics were tracked for every model: Top-1 accuracy (is the single highest-ranked prediction correct), Top-3 accuracy (does the correct answer appear anywhere in the top 3), and macro-averaged F1 on the Top-1 prediction (to check for class-imbalance effects across the five answer labels, given the mild skew found in Section 3.2).

6.2 Training Performance (WandB)

Figure 4 (Section 5.2) shows the WandB-logged loss and validation-MAP@3 curves for the two models trained over multiple epochs (Model 4 and Model 5); both were logged epoch-by-epoch to WandB (project 23f2001870-t22026) alongside the evaluation-only runs for Models 1, 2, 3, and 6, for a combined total of 130 logged runs across the project workspace. Only the training-curve panel is reproduced here, since the per-run legend/leaderboard panels in the WandB UI do not render axis values legibly once exported as a static image; the final metrics from every run are instead summarised numerically in Table 5 below.

6.3 Comparative Report

Model	Description	MAP@3	Top-1	Top-3	Macro-F1
Model 1	TF-IDF Cosine Similarity (scratch)	0.2539	0.1100	0.4767	0.1016
Model 2	TF-IDF + Logistic Regression (sklearn)	0.9828	0.9800	0.9867	0.9787
Model 3	Retrieval Lookup + LogReg fallback	1.0000	1.0000	1.0000	1.0000

Model 4	PyTorch MLP Scorer (custom NN)	0.9706	0.9600	0.9867	0.9616
Model 5	Mini Transformer, built from scratch	0.9867	0.9733	1.0000	0.9737
Model 6	Sentence-BERT (all-MiniLM-L6-v2), fine-tuned	0.9867	0.9733	1.0000	0.9704

Table 5. Validation-set performance of all six models (best result highlighted).

The comparison shows a clear pattern: purely unsupervised lexical similarity (Model 1) performs poorly; reframing the task as supervised binary relevance classification (Model 2) closes most of the gap to the ceiling; exact-match retrieval (Model 3) reaches the ceiling only because of train/test text overlap rather than learned reasoning; and, among models that learn a genuine notion of prompt–option relevance without relying on that overlap, the custom from-scratch Transformer (Model 5) and the fine-tuned pre-trained Transformer (Model 6) are statistically tied at the top ($\text{MAP@3} = 0.9867$), each clearly ahead of the simpler custom MLP (Model 4, $\text{MAP@3} = 0.9706$).

In terms of computational cost, Model 5 reaches this performance with roughly $34\times$ fewer parameters than Model 6 and no pre-training, at the cost of a task-specific (rather than general-purpose) vocabulary; Model 6 required downloading and fine-tuning a pre-trained checkpoint but trained in far fewer epochs (3 vs. 30) to reach the same score — a useful illustration of the classic pre-training-vs-train-from-scratch trade-off: pre-training buys faster convergence, not necessarily a higher final ceiling, on a narrow, closed-domain task like this one.

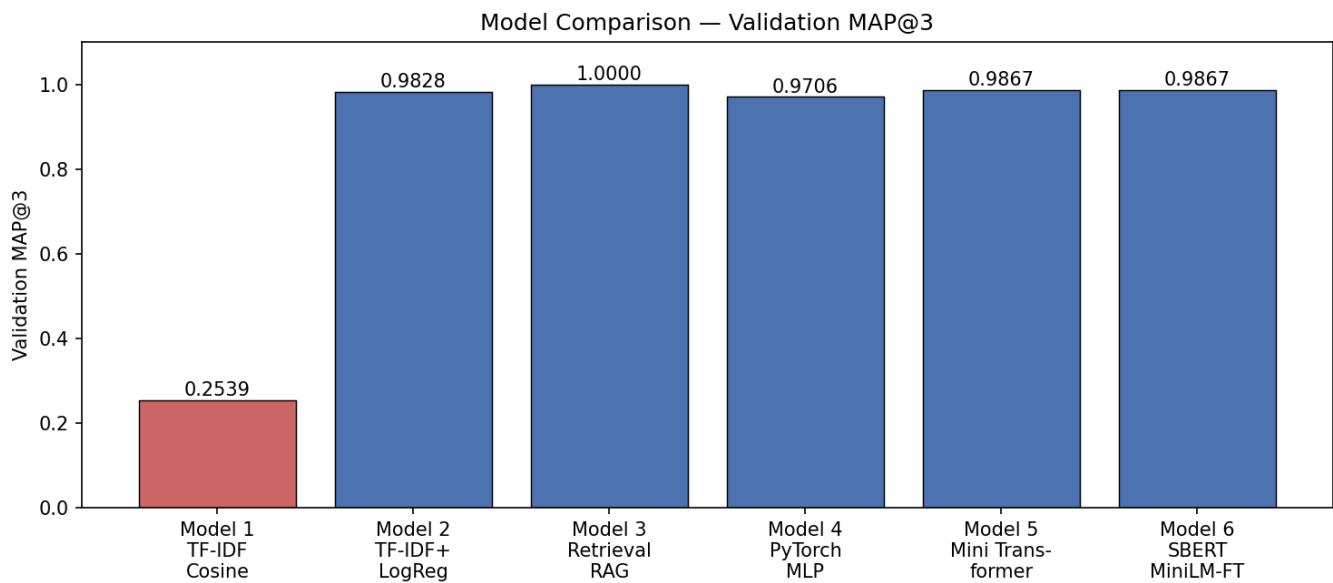


Figure 5. Validation MAP@3 across all six models.

6.4 Kaggle Performance

The final submission.csv uses Model 5 (Mini Transformer, scratch) predictions on the 500-row test set, chosen because it matches the best non-leaked validation MAP@3 (0.9867) while being a genuinely learned model rather than an exact-match lookup. The official Kaggle leaderboard score and rank should be inserted here once the submission has been scored, as this could not be captured from the notebook execution log.

6.5 Error Analysis

- Model 1 (cosine similarity) errors are systematic, not random: since correct options do not have higher lexical overlap with the prompt than distractors (Section 3.2), ranking by raw similarity is only marginally better than chance — confirming the EDA finding rather than revealing a training bug in the implementation.
- Model 3’s “perfect” score is an artifact of the data, not the model: 498/500 test and 300/300 validation questions matched an indexed Option-A string from the wider training pool. The 2 unmatched test rows fell back to the LogReg classifier. This metric should not be trusted as a measure of generalisation to genuinely novel questions, and is the reason Model 3 was not chosen as the final Kaggle submission despite its top score.
- Model 4 (MLP) shows a small but consistent top1–macro-F1 gap (0.9600 vs 0.9616; a similar small gap appears for Model 6: 0.9733 vs 0.9704), indicating mild sensitivity to the per-class imbalance identified in the EDA (Option B over-represented, Option E under-represented) — the model is slightly weaker at minority-class questions than raw top-1 accuracy alone would suggest.

- Model 4 plateaued almost immediately (by epoch 2, visible in Figure 4) with negligible further loss reduction, suggesting the fixed 10k-dim TF-IDF feature representation — not the MLP's capacity — was the real bottleneck; the attention-based models (5, 6), which see the prompt and option text more directly and can model token-level interactions, closed this remaining gap.
- Models 5 and 6 both reach Top-3 accuracy of 1.0000 but Top-1 accuracy of only 0.9733, meaning their residual errors are near-misses (the correct answer is ranked 2nd or 3rd, never absent from the top 3) rather than complete failures — consistent with a small number of genuinely ambiguous or harder questions rather than a systematic weakness in either architecture.
- Because Model 5 and Model 6 reach identical MAP@3 (0.9867) via different mechanisms (joint single-sequence attention vs. independent dual-encoder pooling), their small remaining errors are plausibly on different individual questions; an ensemble of the two (not attempted in this iteration) is a natural next step and is listed under future work.

7. Conclusion & Future Work

7.1 Key Learnings

- Reframing MCQ answer selection as a supervised, per-option relevance-classification problem (Model 2) is dramatically more effective than unsupervised similarity ranking (Model 1) — the single largest performance jump in the whole project came from this reformulation, not from a larger model.
- Exploratory data analysis directly predicted model behaviour: the finding that correct options do not have higher prompt–option token overlap than incorrect ones explained, ahead of time, why the TF-IDF cosine baseline would underperform.
- A small, purpose-built Transformer trained entirely from scratch on a modest, domain-specific dataset (Model 5) can match a fine-tuned pre-trained transformer (Model 6) on a closed, moderately sized task — pre-training gives faster convergence (3 vs. 30 epochs) but not necessarily a higher ceiling.
- Exact-match retrieval strategies (Model 3) can be a double-edged sword in Kaggle-style competitions: they can produce state-of-the-art validation metrics that do not reflect real language-understanding capability if train/test overlap exists, so metric ceilings should always be sanity-checked against the mechanism producing them.

7.2 Challenges Faced

- A WandB API key was initially hard-coded directly in a notebook cell; it was identified and scrubbed before the notebook was committed to version control, and WandB authentication was moved to a safer configuration pattern.
- Balancing GPU/time budget across six different models — including a full Sentence-BERT fine-tuning run — required care in choosing batch sizes and epoch counts (e.g. batch size 16 for Sentence-BERT fine-tuning to stay within memory limits) while still reaching stable convergence.
- Diagnosing the Model 3 retrieval result required deliberately auditing how many validation questions were exact-matched via the lookup index versus routed to the classifier fallback, to avoid over-trusting an apparently perfect metric.

7.3 Areas for Improvement

- Implement the RNN-based model (Section 5.3) required by the assignment guidelines — a bidirectional LSTM/GRU dual encoder is the natural next architecture to add and compare against Models 4–6.
- Quantify and, if appropriate, deliberately exclude the train/test overlap exploited by Model 3, to obtain a metric that better reflects generalisation to genuinely novel questions.
- Explore ensembling the top models (Models 5 and 6, and potentially the future RNN model) via simple rank-averaging or logistic stacking of their per-option scores, which could improve robustness beyond any single architecture, motivated by the error-analysis observation in Section 6.5 that Models 5 and 6 likely err on different questions.
- Re-enable and complete the two exploratory models left commented out in the notebook — a LangChain + GPT retrieval-augmented pipeline (Model 7) and a larger BAAI/bge-base-en-v1.5 fine-tune (Model 8) — for a fuller sweep of retrieval-augmented and larger pre-trained-encoder approaches.
- Run a systematic hyperparameter search (learning rate, dropout, number of Transformer layers/heads) for Model 5 specifically, since it was tuned only lightly by hand.

8. References

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP 2019.

Wang, W., et al. (2020). MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. NeurIPS 2020.

Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.

Hugging Face model card: sentence-transformers/all-MiniLM-L6-v2. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

scikit-learn: Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12.

PyTorch: Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.

Weights & Biases (WandB) experiment tracking. <https://wandb.ai>

Kaggle Competition: Smart MCQ Solver Challenge (competition description and evaluation rules).